DOGECOIN PREDICTION ALGORITHM

AI Agent Development Guide — v2.0 (DOGE Edition)

Structure · Best Practices · DOGE-Specific Architecture · Failure Catalogue · Recovery Protocols

Document Purpose & Scope

This is the v2.0 update of the AI Agent Development Guide, fully revised to target DOGECOIN (DOGEUSDT)

on Binance as the exclusive prediction subject. Every section has been updated to reflect DOGE-specific

data characteristics, regime behaviour, feature requirements, known failure modes, and operational rules.

Key additions in v2.0:

- DOGE asset profile and behavioural characteristics (Section 2)
- Full multi-source data extraction plan with exact Binance endpoints (Section 4)
- DOGE market regime classification system (Section 5)
- DOGE-specific feature engineering layer with 12 new mandatory features (Section 7)
- Revised training window recommendation with regime-based sampling rules (Section 8)
- DOGE-specific error catalogue covering sentiment-driven failure modes (Section 12)
- Updated quality gates with DOGE-calibrated performance thresholds (Section 15)
- Revised agent execution checklist (Section 16)

Colour legend used throughout: BLUE = architecture & standards RED = errors & failures

ORANGE = cautions & warnings GREEN = validated best practice

PURPLE = DOGE-specific rules TEAL = regime-related content

Table of Contents

1. Project Architecture & Directory Structure
2. DOGE Asset Profile & Behavioural Characteristics
3. Environment Setup & Dependency Management
4. Data Sources — Full Binance Extraction Plan for DOGE

5. DOGE Market Regime Classification System
6. Data Processing & Storage Layer
7. Feature Engineering — Standard + DOGE-Specific
8. Model Architecture & Training Layer
9. Backtesting & Validation Layer
10. Signal Generation & Inference Layer
11. Deployment & Monitoring Layer
12. Complete Error & Failure Catalogue
13. Anti-Pattern Registry
14. Self-Correction & Recovery Protocols
15. Quality Gates & Acceptance Criteria
16. Agent Execution Checklist

1. Project Architecture & Directory Structure

The agent MUST create and maintain this exact directory structure. DOGE-specific modules are marked with [DOGE]. These are new or significantly modified from the generic framework.

⚠ CRITICAL — Canonical Directory Tree (DOGE Edition)

```
doge_predictor/
├── config/
│   ├── settings.yaml          # All tunable parameters — never hardcode
│   ├── doge_settings.yaml     # [DOGE] DOGE-specific overrides & thresholds
│   ├── regime_config.yaml     # [DOGE] Regime classification thresholds
│   ├── secrets.env            # API keys — NEVER commit to version control
│   └── logging.yaml
├── data/
│   ├── raw/
│   │   ├── dogeusdt_1h/       # [DOGE] Primary: DOGEUSDT 1h candles (2019-present)
│   │   ├── dogeusdt_4h/       # [DOGE] Higher-TF features source
│   │   ├── dogeusdt_1d/       # [DOGE] Daily trend context
│   │   ├── dogebtc_1h/        # [DOGE] BTC-relative strength data
│   │   ├── btcusdt_1h/        # [DOGE] BTC correlation feature source
│   │   ├── funding_rates/     # [DOGE] DOGE perpetual futures funding rates
│   │   └── agg_trades/        # [DOGE] Recent 30-day aggregate trade data
│   ├── processed/
│   │   ├── dogeusdt_1h_clean/  # Validated, gap-filled primary data
│   │   ├── dogebtc_1h_clean/
│   │   └── btcusdt_1h_clean/
│   ├── features/
│   │   ├── primary/           # Full feature matrix (post-2022 training set)
│   │   ├── context/           # Pre-2022 context features (ATH distance, LT trend)
│   │   └── live/              # Rolling live feature buffer
│   ├── regimes/               # [DOGE] Regime labels and transition log
│   └── checkpoints/           # Bootstrap progress checkpoints
├── src/
│   ├── ingestion/
│   │   ├── rest_client.py     # Binance REST API wrapper (spot)
│   │   ├── futures_client.py  # [DOGE] Binance Futures API (funding rates)
│   │   ├── ws_client.py       # Binance WebSocket client
│   │   ├── bootstrap.py       # Historical data backfill
│   │   ├── multi_symbol.py    # [DOGE] Parallel ingestion: DOGE+BTC+DOGEBTC
│   │   └── scheduler.py       # Incremental update scheduler
│   ├── processing/
│   │   ├── cleaner.py
│   │   ├── validator.py
```

```
│   │   ├── aligner.py          # [DOGE] Align DOGE, BTC, DOGEBTC to same timeline
│   │   └── storage.py
│   ├── regimes/                # [DOGE] Full regime classification subsystem
│   │   ├── __init__.py
│   │   ├── classifier.py       # Assigns regime label to each candle
│   │   ├── detector.py         # Real-time regime change detection
│   │   └── features.py         # Regime-derived features for model input
│   ├── features/
│   │   ├── price_indicators.py
│   │   ├── volume_indicators.py
│   │   ├── orderbook_features.py
│   │   ├── lag_features.py
│   │   ├── doge_specific.py      # [DOGE] BTC correlation, DOGE/BTC ratio, vol spike
│   │   ├── funding_features.py   # [DOGE] Funding rate signal engineering
│   │   ├── htf_features.py       # [DOGE] Higher-timeframe (4h, 1d) derived features
│   │   └── pipeline.py
│   ├── models/
│   │   ├── base_model.py
│   │   ├── lstm_model.py
│   │   ├── xgb_model.py
│   │   ├── transformer_model.py
│   │   ├── ensemble.py
│   │   └── regime_router.py      # [DOGE] Routes to regime-specific model weights
│   ├── training/
│   │   ├── walk_forward.py
│   │   ├── trainer.py
│   │   ├── regime_trainer.py     # [DOGE] Per-regime model training
│   │   ├── hyperopt.py
│   │   └── scaler.py
│   ├── evaluation/
│   │   ├── metrics.py
│   │   ├── backtest.py
│   │   └── reporter.py
│   ├── inference/
│   │   ├── engine.py
│   │   └── signal.py
│   └── monitoring/
│       ├── drift_detector.py
│       ├── regime_monitor.py     # [DOGE] Monitors regime transitions in live trading
│       ├── health_check.py
│       └── alerting.py
├── tests/
│   ├── unit/
│   │   ├── test_doge_features.py  # [DOGE] Mandatory DOGE-specific feature tests
```

```
│   │   ├── test_regime_classifier.py
│   │   └── test_alignment.py      # [DOGE] Multi-symbol timestamp alignment tests
│   ├── integration/
│   └── fixtures/
│       └── doge_sample_data/      # [DOGE] Realistic DOGE fixture data for all regimes
├── mlruns/
├── notebooks/
│   ├── doge_eda.ipynb             # [DOGE] Exploratory analysis — scratch only
│   └── regime_analysis.ipynb      # [DOGE] Regime visualisation — scratch only
├── scripts/
│   ├── bootstrap_doge.py          # [DOGE] Full multi-symbol bootstrap runner
│   ├── label_regimes.py           # [DOGE] One-time regime labelling on full history
│   ├── train.py
│   └── serve.py
└── docker-compose.yml
```

DIRECTORY VIOLATIONS — Agent Must Never Do These

- NEVER store DOGEBTC data in the DOGEUSDT raw folder — different pair, different schema
- NEVER mix funding rate data (futures) with spot OHLCV data in the same table
- NEVER write to data/raw/ after bootstrap — it is immutable and append-only
- NEVER import from notebooks/ — they are scratch space and contain no production logic
- NEVER hardcode 'DOGEUSDT' as a string in src/ — always read from config/doge_settings.yaml
- NEVER store pre-2022 training rows in the same Parquet partition as post-2022 rows (different statistical properties require explicit partitioning by era)

2. DOGE Asset Profile & Behavioural Characteristics

This section is mandatory reading before any model design decisions are made. DOGE behaves differently from BTC, ETH, and most other assets. Ignoring these characteristics produces models that fail silently in production.

DOGE Asset Identity

Exchange Pair (Primary):	DOGEUSDT — Binance Spot
Exchange Pair (Secondary):	DOGEBTC — Binance Spot (relative strength signal)
Exchange Pair (Tertiary):	DOGEBNB — Binance Spot (low priority, low volume)
Futures Pair:	DOGEUSDT — Binance USD-M Perpetual Futures
Data Available From:	~July 2019 (Binance listing)
Primary Training Window:	January 2022 — Present (post-mania normalization)
Context Window:	July 2019 — December 2021 (stored, not primary training)
Primary Interval:	1 hour (recommended) — ~26,000 candles from Jan 2022

Secondary Intervals:	4h, 1d — for higher-timeframe feature derivation only

2.1 Three Historical Regimes

DOGE's full history contains three statistically distinct regimes. Each has different volatility, correlation, and predictability properties. The model must know which regime it is in.

DOGE Historical Regime Map

Regime	Period	Price Range	Characteristics	Model Treatment
DORMANT	Jul 2019 – Dec 2020	$0.002 – $0.007	Ultra-low volatility. Tight range. BTC-correlated. Very low volume. Predictable but low edge.	Store as context only. Do NOT train primary model on this era.
MANIA	Jan 2021 – May 2021	$0.007 – $0.731	Parabolic 100x move. Sentiment-driven. Fundamentals irrelevant. Extreme volume. Decoupled from BTC.	Exclude from primary training. Use only for extreme-regime feature calibration.
NORMALIZATION	Jun 2021 – Dec 2021	$0.700 – $0.150	Post-peak distribution. High volatility. Capitulation patterns. Transitional regime.	Exclude from primary training. Too transitional for stable feature statistics.
CURRENT	Jan 2022 – Present	$0.060 – $0.220	Moderate volatility. BTC-correlated with periodic decoupling. Consistent market structure.	PRIMARY TRAINING DATASET. All model training uses this era only.

2.2 DOGE-Specific Behavioural Rules

Mandatory Behavioural Facts — Agent Must Internalize

FACT 1 — SENTIMENT PRIMACY: DOGE price is more sensitive to social media activity and celebrity mentions than to on-chain metrics or technical levels. Technical models will miss sentiment-driven moves. Mitigation: use volume spike detection as a sentiment proxy (highest available signal).

FACT 2 — BTC BETA: DOGE has a high rolling BTC beta (~0.8-1.2 in the current regime). When BTC
sells off, DOGE sells off harder. When BTC rallies, DOGE may rally more or may lag.
Mitigation: always include BTC return and BTC volatility as explicit model features.

FACT 3 — DECOUPLING EVENTS: Periodically DOGE decouples from BTC (rolling 24h corr drops < 0.3).
These decoupling events are among the highest-signal moments for DOGE-specific prediction.
Mitigation: rolling BTC-DOGE correlation is a mandatory feature.

FACT 4 — VOLUME AS LEADING INDICATOR: A DOGE volume spike (> 3x 20-period average) has
historically preceded continued directional movement within 4-12 hours. Volume leads price.
Mitigation: volume_spike_flag and volume_ratio are mandatory features.

FACT 5 — FUNDING RATE EXTREMES: DOGE perpetual futures funding rate > +0.1% per 8h signals
extreme long crowding → mean reversion likely. Rate < -0.05% signals extreme short crowding
→ short squeeze potential. These are asymmetric signals — strong in extremes, noise in middle.
Mitigation: funding rate and funding rate z-score are mandatory features.

FACT 6 — ALTCOIN SEASON SENSITIVITY: DOGE outperforms strongly during altcoin seasons.
DOGE/BTC ratio rising while BTC dominance falls = strongest bullish regime for DOGE.
Mitigation: DOGE/BTC momentum and BTC dominance proxy (BTC return vs DOGE return spread)
are mandatory features.

FACT 7 — ROUND NUMBER PSYCHOLOGY: DOGE retail traders fixate on round numbers ($0.10, $0.20,
$0.50). Price tends to stall and reverse at these levels more than other assets.
Mitigation: distance_to_round_number feature captures this psychological dynamic.

3. Environment Setup & Dependency Management

3.1 Python Version & Virtual Environment

Use Python 3.11 or 3.12 exclusively. Create an isolated virtual environment before any installation.

✓Environment Bootstrap

```
python3.11 -m venv .venv
```

```
source .venv/bin/activate
pip install --upgrade pip setuptools wheel
pip install -r requirements.txt
pip install -r requirements-dev.txt
```

Core Dependencies — Exact Pinned Versions	
python-binance	1.0.19 — Spot REST + WebSocket. Futures client built separately.
pandas	2.2.x — Use copy-on-write mode: pd.options.mode.copy_on_write = True
numpy	1.26.x
torch	2.3.x — LSTM + Transformer. Specify +cpu or +cu121 explicitly.
xgboost	2.0.x
scikit-learn	1.4.x
ta-lib	0.4.x — Requires TA-Lib C library at OS level first
pandas-ta	0.3.x — Pure-Python fallback; use for funding rate indicators
mlflow	2.13.x
optuna	3.6.x
sqlalchemy	2.0.x — TimescaleDB ORM
psycopg2-binary	2.9.x — PostgreSQL/TimescaleDB driver
pydantic	2.7.x — Config validation + data contracts
loguru	0.7.x — Structured logging throughout
pandera	0.19.x — [DOGE] Schema enforcement on DataFrames
filelock	3.13.x — Concurrent write protection
scipy	1.13.x — Statistical tests (ADF, correlation)
pytest	8.x — Testing
prometheus-client	0.20.x

DOGE-Specific Dependency Issues

- python-binance Futures API: the Client class handles both spot and futures, but funding rate endpoints require futures_* method prefix. Test with: client.futures_funding_rate(symbol='DOGEUSDT')
- Funding rate data is only available from ~Oct 2020. Do not request before that date.
- pandera is newly required in v2.0 — add to requirements.txt before any data processing runs.
- scipy ADF test (statsmodels.tsa.stattools.adfuller) requires statsmodels: add statsmodels==0.14.x

4. Data Sources — Full Binance Extraction Plan for DOGE

This section defines every data source, the exact Binance endpoint, the extraction priority, the date range, and the storage destination. The agent must execute these in the priority order listed.

Data Extraction Priority Order

Priority 1 — DOGEUSDT 1h candles (Jul 2019 – present) → data/raw/dogeusdt_1h/
Priority 2 — BTCUSDT 1h candles (Jul 2019 – present) → data/raw/btcusdt_1h/
Priority 3 — DOGEBTC 1h candles (Jul 2019 – present) → data/raw/dogebtc_1h/
Priority 4 — DOGEUSDT 4h candles (Jul 2019 – present) → data/raw/dogeusdt_4h/
Priority 5 — DOGEUSDT 1d candles (Jul 2019 – present) → data/raw/dogeusdt_1d/
Priority 6 — DOGE Funding Rates (Oct 2020 – present) → data/raw/funding_rates/
Priority 7 — DOGEUSDT Agg Trades (last 30 days only) → data/raw/agg_trades/

BTC data (Priority 2) is extracted in PARALLEL with DOGE data — not after.
Both must complete before any feature engineering begins.

Data Source Specifications

Source	Binance Endpoint	Interval	Date Range	Est. Rows	Priority
DOGEUSDT Spot 1h	GET /api/v3/klines?symbol=DOGEUSDT&interval=1h	1 hour	Jul 2019 – now	~42,000	1 — CRITICAL
BTCUSDT Spot 1h	GET /api/v3/klines?symbol=BTCUSDT&interval=1h	1 hour	Jul 2019 – now	~42,000	2 — CRITICAL
DOGEBTC Spot 1h	GET /api/v3/klines?symbol=DOGEBTC&interval=1h	1 hour	Jul 2019 – now	~42,000	3 — HIGH
DOGEUSDT Spot 4h	GET /api/v3/klines?symbol=DOGEUSDT&interval=4h	4 hours	Jul 2019 – now	~10,500	4 — MEDIUM
DOGEUSDT Spot 1d	GET /api/v3/klines?symbol=DOGEUSDT&interval=1d	1 day	Jul 2019 – now	~1,750	5 — MEDIUM
DOGE Funding Rate	GET /fapi/v1/fundingRate?symbol=DOGEUSDT	8 hours	Oct 2020 – now	~5,400	6 — HIGH
DOGEUSDT Agg Trades	GET /api/v3/aggTrades?symbol=DOGEUSDT	Tick	Last 30 days	~5M+	7 — LOW (live only)

4.1 Bootstrap Execution Rules

✅Multi-Symbol Bootstrap Protocol

RULE 1 — Run DOGEUSDT 1h and BTCUSDT 1h bootstraps concurrently (separate threads/processes).
DOGEBTC 1h runs after both complete to avoid rate limit conflicts.

RULE 2 — Pagination: fetch exactly 1000 candles per request. Use close_time of last row as startTime of next request. Pause 100ms between requests.

RULE 3 — Save checkpoint every 5,000 rows: {symbol, interval, last_open_time, row_count}.
On restart, read checkpoint and continue from last_open_time.

RULE 4 — After all bootstrap completes, run the aligner (src/processing/aligner.py) to verify DOGE, BTC, and DOGEBTC all share exactly the same timestamp index.
Any symbol with gaps that others do not have must be individually backfilled.

RULE 5 — Funding rate bootstrap uses GET /fapi/v1/fundingRate with limit=1000.
Funding rates are emitted every 8 hours. Validate: interval between consecutive rows is exactly 28800000ms (8h). Any deviation is a data error.

RULE 6 — Agg trades are NOT bootstrapped historically. They are collected in rolling 30-day windows for live order flow features only. Historical agg trade data is too large to store and too granular to use in 1h model training.

4.2 Data Alignment Contract

All three primary symbols (DOGEUSDT, BTCUSDT, DOGEBTC) must share an identical timestamp index before features are computed. This is enforced by the aligner module.

□ Multi-Symbol Alignment Errors

- DOGEBTC occasionally has maintenance gaps that DOGEUSDT does not have.
  The aligner must detect these and forward-fill DOGEBTC price ONLY (not volume) for gaps <= 3 candles.
  For gaps > 3 candles in DOGEBTC, tag those rows as dogebtc_interpolated=True.
  DOGE relative-strength features computed during these gaps must be masked as NaN.

- Never outer-join the three DataFrames and assume NaN == missing.
  Always inner-join first to find the common date range, then investigate any symbol

that has fewer rows than the intersection to identify and recover specific gaps.

- BTC data is used only as a feature source — never as a training target.
  Keep BTC data in a separate storage table. Never merge BTC OHLCV columns into the DOGE OHLCV table. Compute cross-asset features in features/doge_specific.py instead.

5. DOGE Market Regime Classification System

Regime classification is mandatory for DOGE. A model trained without regime awareness will fail during regime transitions — which are the highest-volatility, highest-risk periods. The regime system must be built before any model training begins.

5.1 Regime Definition

DOGE Regime Taxonomy (Post-2022 Current Regime)

Within the CURRENT regime (Jan 2022 – present), the following sub-regimes are classified:

SUB-REGIME 1 — TRENDING_BULL
Definition: 20-period EMA > 50-period EMA > 200-period EMA. Price above all three.
ATR(14) normalized > 0.4% per candle. Rolling 7d return > +5%.
Model Behaviour: Momentum features weighted higher. Mean-reversion features suppressed.

SUB-REGIME 2 — TRENDING_BEAR
Definition: 20-period EMA < 50-period EMA < 200-period EMA. Price below all three.
ATR(14) normalized > 0.4% per candle. Rolling 7d return < -5%.
Model Behaviour: Momentum features weighted higher (bearish direction). RSI oversold ignored.

SUB-REGIME 3 — RANGING_HIGH_VOL
Definition: Price oscillates between 20-period Bollinger Bands. ATR > 0.5% per candle.
BB width > 0.06. Rolling 7d return between -5% and +5%.
Model Behaviour: Mean-reversion signals active. Momentum signals suppressed.

SUB-REGIME 4 — RANGING_LOW_VOL
Definition: BB width < 0.04. ATR < 0.3% per candle. Rolling 7d return near zero.
Model Behaviour: Low confidence → reduce position sizing by 50%. Increase threshold to 0.70.

SUB-REGIME 5 — DECOUPLED (DOGE-SPECIFIC)
Definition: Rolling 24h correlation between DOGE and BTC returns < 0.30.

This is a DOGE-specific regime that overrides all others when active.
Model Behaviour: Suppress BTC-correlated features. Amplify DOGE-specific features.
Increase confidence threshold to 0.72. Maximum alert state.

5.2 Regime Classification Implementation

✅Regime Classifier — Required Logic Skeleton

```
class DogeRegimeClassifier:
    def classify(self, df: pd.DataFrame) -> pd.Series:
        '''
        Input:  DataFrame with OHLCV + BTC returns aligned by timestamp
        Output: Series of regime labels indexed by timestamp
        '''
        regimes = pd.Series(index=df.index, dtype='str')

        # Step 1: Compute classification inputs
        ema20  = df['close'].ewm(span=20).mean()
        ema50  = df['close'].ewm(span=50).mean()
        ema200 = df['close'].ewm(span=200).mean()
        atr_norm = compute_atr(df, 14) / df['close']
        bb_width = compute_bb_width(df, 20, 2)
        roll7d_ret = df['close'].pct_change(168)  # 168 x 1h = 7 days
        btc_corr = df['doge_return'].rolling(24).corr(df['btc_return'])

        # Step 2: Apply DECOUPLED check first (highest priority override)
        regimes[btc_corr < 0.30] = 'DECOUPLED'

        # Step 3: Apply trend regimes
        bull_trend = (ema20 > ema50) & (ema50 > ema200) & (roll7d_ret > 0.05)
        bear_trend = (ema20 < ema50) & (ema50 < ema200) & (roll7d_ret < -0.05)
        regimes[bull_trend & (regimes != 'DECOUPLED')] = 'TRENDING_BULL'
        regimes[bear_trend & (regimes != 'DECOUPLED')] = 'TRENDING_BEAR'

        # Step 4: Fill remaining with ranging regimes
        ranging = ~bull_trend & ~bear_trend & (regimes != 'DECOUPLED')
        regimes[ranging & (bb_width > 0.04)] = 'RANGING_HIGH_VOL'
        regimes[ranging & (bb_width <= 0.04)] = 'RANGING_LOW_VOL'

        return regimes
```

□ Regime-Based Training Rules

RULE 1 — Assign regime label to every row in the training dataset before model training.
RULE 2 — Train a SEPARATE XGBoost model for each of the 5 sub-regimes.
The LSTM is trained on all regimes combined (it learns transitions).
The ensemble meta-learner receives: [lstm_prob, xgb_prob, regime_label_encoded].
RULE 3 — At inference time, classify the current regime BEFORE running model inference.
Route the signal through the appropriate regime-specific XGBoost model.
RULE 4 — When regime is RANGING_LOW_VOL: reduce position size by 50% regardless of signal.
RULE 5 — When regime is DECOUPLED: increase confidence threshold to 0.72 and emit alert.
RULE 6 — Log every regime transition with timestamp. Track regime duration distribution.
RULE 7 — Regime classifier must be re-run on live data every candle close.
Regime label changes must be logged and do not require model retraining.

6. Data Processing & Storage Layer

6.1 Validation Contract

Every data batch must pass all validation checks before writing to processed storage. Validation failures halt the pipeline — they are never silently skipped.

✅Full Validation Checklist	
Schema Check	Columns: open_time, open, high, low, close, volume, close_time, quote_volume, num_trades. Reject any batch with missing or extra columns.
Timestamp Monotonic	df['open_time'].is_monotonic_increasing must be True. Reject entire batch if False.
Interval Consistency	Gap between consecutive open_times must equal interval_ms exactly. Log any violation. Gap > 3 candles = pipeline halt.
OHLCV Sanity	high >= open, high >= close, low <= open, low <= close, high >= low, close > 0, volume >= 0. Tag violating rows as corrupted. Do not delete — log for investigation.
No NaN / Inf	After cleaning, zero NaN or Inf in any OHLCV column. This is a hard requirement.
Multi-Symbol Alignment	After aligner runs: assert DOGEUSDT, BTCUSDT, DOGEBTC all have identical timestamp index.
Funding Rate Interval	Funding rates: consecutive rows must be exactly 28800000ms (8h) apart. Any other gap is a data error.
Era Partitioning	Pre-2022 rows stored with era='context'. Post-2022 rows stored with era='training'. Never mixed in same partition.
Timezone Enforcement	ALL timestamps stored as UTC epoch milliseconds. Reject any tz-naive datetime object at validator boundary.

Stale Data Check	Last candle close_time must be within 2 * interval_ms of current UTC before inference. Halt if stale.

6.2 Storage Architecture

Storage Layer Design	
Primary OHLCV	TimescaleDB — separate hypertable per symbol+interval. Partition by month. Index on open_time.
Funding Rates	TimescaleDB — funding_rates hypertable. Columns: timestamp_ms, symbol, funding_rate, mark_price.
Feature Store	Apache Parquet — partitioned by era (pre-2022/post-2022) and by regime label. Versioned by run_id.
Regime Labels	Parquet file + TimescaleDB table: timestamp_ms, regime_label, btc_corr_24h, bb_width, atr_norm.
Model Registry	MLflow artifact store — model weights + scaler.pkl + feature_columns.json + regime_config.yaml.
Signal Log	Append-only TimescaleDB table — all fields from Section 10 inference protocol.
Live Feature Buffer	In-memory ring buffer (collections.deque, maxlen=500) — last 500 closed candles for real-time inference.

□ Data Processing Errors			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**
Forward-fill on OHLCV price gaps	cleaner.py	Synthetic prices enter training data	Do not forward-fill price. Tag as interpolated. Mask in feature computation.
Outer join producing phantom rows	aligner.py	NaN rows treated as real data in features	Always inner-join on timestamp. Log which symbol caused the mismatch.
Funding rate timezone mismatch	ingestion/futures_client.py	Funding rates misaligned by 8h with OHLCV	Normalize funding rate timestamps to candle open_time of the 1h candle they fall within.
Era mixing in Parquet partitions	processing/storage.py	Pre-2022 mania stats corrupt post-2022 scaler	Enforce era column. Use partition_by=['era'] in Parquet write. Assert in validator.
Schema drift between Parquet writes	processing/storage.py	Column mismatch crash at training	Use pandera schema validation on every write. Store schema hash per run_id.

7. Feature Engineering — Standard + DOGE-Specific

Feature engineering for DOGE requires two layers: the standard technical indicator set (applicable to any asset) and the DOGE-specific feature set (applicable only to DOGE). Both layers are mandatory.

7.1 Standard Technical Features

Price-Based Indicators	
SMA / EMA	Periods: 7, 14, 21, 50, 200. Compute on close. Also compute: close/EMA200 ratio (distance from long-term trend).
MACD	12/26/9. Features: macd_line, signal_line, histogram, histogram_direction (sign change flag).
RSI	Period 14. Features: rsi_value, rsi_overbought (>70), rsi_oversold (<30), rsi_divergence (price vs RSI direction).
Bollinger Bands	20-period, 2 std. Features: bb_upper, bb_lower, bb_pct_b, bb_width, bb_squeeze_flag (width < 0.04).
ATR	Period 14. Store as raw ATR and ATR normalized (ATR/close). Use normalized ATR for all comparisons.
Stochastic	14/3/3. Features: stoch_k, stoch_d, stoch_crossover_flag.
Ichimoku	Tenkan(9), Kijun(26), Senkou A, Senkou B. Feature: cloud_position (above/below/inside cloud = 1/−1/0).

Volume-Based Indicators	
OBV	On-Balance Volume. Also compute: obv_ema_ratio (OBV / 20-period EMA of OBV).
VWAP	Reset at UTC midnight daily. Feature: price_vs_vwap (close - VWAP) / VWAP.
Volume MA Ratio	volume / volume.rolling(20).mean(). This is the base for volume_spike_flag.
CMF	Chaikin Money Flow, 20-period.
CVD	Cumulative Volume Delta. Requires agg_trade stream for live. For historical: approximate using (close-low)/(high-low) * volume.

Lag & Rolling Features

Log Returns	log(close[t]/close[t-n]) for n = 1, 3, 6, 12, 24, 48, 168 (7d). These are the primary price-change inputs.
Rolling Volatility	std(log_returns) over windows: 6, 12, 24, 48, 168. Normalize each by its own rolling mean.
Rolling Skewness / Kurtosis	24-period window. Kurtosis spike often precedes fat-tail moves in DOGE.
Price Momentum	close[t] / close[t-n] - 1 for n = 6, 12, 24, 48. Captures trend strength.
High-Low Range	(high - low) / close. Normalized intrabar range — proxy for indecision.

7.2 DOGE-Specific Mandatory Features

The following 12 features are mandatory for DOGE. They must be computed in src/features/doge_specific.py and src/features/funding_features.py. A model trained without these features is considered incomplete.

□ DOGE-Specific Feature #1 — BTC Rolling Correlation (MANDATORY)

Module: src/features/doge_specific.py
Formula: doge_btc_corr_N = doge_log_returns.rolling(N).corr(btc_log_returns)
Windows: N = 12 (12h), N = 24 (1d), N = 168 (7d)
Output features: doge_btc_corr_12h, doge_btc_corr_24h, doge_btc_corr_7d

Interpretation:
corr > 0.7 → DOGE is BTC-correlated; BTC direction is predictive for DOGE
corr < 0.3 → DOGE is DECOUPLED; regime override triggered
corr < 0.0 → DOGE is inversely correlated with BTC; extreme rare state

Error to avoid: using raw price correlation instead of log-return correlation.
Raw price correlation is spurious (non-stationary). Always use log-returns.

□ DOGE-Specific Feature #2 — DOGE/BTC Ratio Momentum (MANDATORY)

Module: src/features/doge_specific.py
Inputs: DOGEBTC 1h close price (from data/raw/dogebtc_1h/)
Formula: dogebtc_momentum_N = log(dogebtc_close[t] / dogebtc_close[t-N])
Windows: N = 6, 24, 48
Output features: dogebtc_mom_6h, dogebtc_mom_24h, dogebtc_mom_48h

Interpretation:
Positive → DOGE is outperforming BTC in BTC terms → altcoin season signal
Negative → DOGE is underperforming BTC → BTC accumulation phase, DOGE may lag

Error to avoid: computing this ratio from DOGEUSDT/BTCUSDT (introduces USD noise).

Always use the dedicated DOGEBTC pair price directly.

□ DOGE-Specific Feature #3 — Volume Spike Detection (MANDATORY)

Module: src/features/doge_specific.py

Formula: volume_ratio = volume / volume.rolling(20).mean()

```
volume_spike_flag = (volume_ratio >= 3.0).astype(int)
volume_spike_magnitude = volume_ratio.clip(upper=10.0) / 10.0
```

Output features: volume_ratio, volume_spike_flag, volume_spike_magnitude

Interpretation:

- volume_spike_flag == 1 AND price rising → likely continuation within 4-12h
- volume_spike_flag == 1 AND price falling → possible capitulation or dumping
- volume_spike_flag with neutral price → accumulation or distribution

Error to avoid: computing volume spike on raw volume without rolling mean normalization.
DOGE volume trends upward over time; raw volume is non-stationary.

□ DOGE-Specific Feature #4 — Funding Rate Signal (MANDATORY)

Module: src/features/funding_features.py

Inputs: data/raw/funding_rates/ — DOGEUSDT perpetual, 8h frequency

Step 1 — Align: forward-fill funding rate to 1h OHLCV index
(each 8h funding rate applies to the 8 candles it covers)

Step 2 — Compute rolling z-score: funding_rate_zscore = (fr - fr.rolling(90).mean()) / fr.rolling(90).std()
Use 90-period window = ~30 days of 8h funding rates

Step 3 — Compute extreme flags:

```
funding_extreme_long  = (funding_rate > 0.001).astype(int)  # > 0.1% per 8h
funding_extreme_short = (funding_rate < -0.0005).astype(int)  # < -0.05% per 8h
```

Output features: funding_rate, funding_rate_zscore, funding_extreme_long, funding_extreme_short

Error to avoid: using raw funding rate without z-score normalization.
The baseline funding rate drifts over time; raw values are not comparable across regimes.
Also: funding rate data only exists from Oct 2020. For pre-Oct-2020 rows, set all funding features to 0.0 and add a funding_available flag = 0 for those rows.

□ DOGE-Specific Feature #5 — Higher-Timeframe Context (MANDATORY)

Module: src/features/htf_features.py

Inputs: data/raw/dogeusdt_4h/ and data/raw/dogeusdt_1d/

4H Features (merged into 1h frame by forward-filling):

```
  htf_4h_rsi     — RSI(14) on 4h closes
  htf_4h_trend   — 4h EMA20 > EMA50 → 1; else → -1
  htf_4h_bb_pctb — 4h Bollinger %B position

1D Features (merged into 1h frame by forward-filling):
  htf_1d_trend   — 1d EMA20 > EMA50 → 1; else → -1
  htf_1d_return  — 1d log return
  ath_distance   — log(ath_price / close) — distance from all-time high
                   (ATH = $0.731 from May 2021; update if exceeded)

Error to avoid: using future 4h/1d candle values at 1h inference time (lookahead).
Forward-fill HTF features using the LAST CLOSED 4h/1d candle at each 1h step.
A 4h candle that closes at 16:00 is not available until 16:01 — respect this boundary.
```

DOGE-Specific Feature #6 — Round Number Distance (MANDATORY)

```
Module:  src/features/doge_specific.py
Round number levels for DOGE: [0.05, 0.10, 0.15, 0.20, 0.25, 0.30, 0.50, 1.00]

Formula:
  nearest_round = min(round_levels, key=lambda r: abs(close - r))
  distance_to_round_pct = (close - nearest_round) / nearest_round
  at_round_number_flag = (abs(distance_to_round_pct) < 0.01).astype(int)

Output features: distance_to_round_pct, at_round_number_flag, nearest_round_level

Note: This feature requires updating if DOGE price exits the current range.
      Add round levels dynamically based on current price decade.
```

DOGE-Specific Features Summary — All 12	
doge_btc_corr_12h / 24h / 7d	Rolling BTC-DOGE log-return correlation at 3 windows
dogebtc_mom_6h / 24h / 48h	DOGEBTC pair log-return momentum — altcoin season proxy
volume_ratio	Current volume / 20-period rolling mean volume
volume_spike_flag	Binary: 1 if volume > 3x rolling mean
volume_spike_magnitude	Clipped normalized spike size [0,1]
funding_rate	8h DOGE perpetual funding rate (forward-filled to 1h)
funding_rate_zscore	90-period rolling z-score of funding rate
funding_extreme_long	Binary: 1 if funding > 0.001 (crowded longs)
funding_extreme_short	Binary: 1 if funding < -0.0005 (crowded shorts)

htf_4h_rsi / htf_4h_trend / htf_4h_bb_pctb	4h timeframe indicator features
htf_1d_trend / htf_1d_return / ath_distance	Daily trend, return, and ATH proximity
distance_to_round_pct / at_round_number_flag / nearest_round_level	Round number psychology features

□ Feature Engineering Errors

Error / Mistake	Where It Occurs	Consequence	How to Detect & Fix
Lookahead: using close[t] as feature AND target	features/pipeline.py	Perfect in-sample prediction; zero live edge	Target = close.pct_change().shift(-1) > 0. Features use only data available at candle T close.
BTC correlation on raw prices (not returns)	features/doge_specific.py	Spurious non-stationary correlation; meaningless signal	Compute correlation on log returns only. Write unit test: assert corr(price, price) != corr(ret, ret).
HTF candle lookahead (4h candle not yet closed)	features/htf_features.py	Feature uses future data; inflated backtest performance	At 1h candle 15:00, the current 4h candle closes at 16:00. Use the 4h candle that closed at 12:00.
Funding rate not forward-filled to 1h index	features/funding_features.py	NaN in funding features for 7/8 candles in each 8h period	Use df.resample('1h').ffill() to align funding rate to 1h OHLCV index.
Volume spike on absolute volume (non-stationary)	features/doge_specific.py	High values in high-volume eras treated as spikes even if normal for that period	Always normalize: volume / rolling_mean_volume. Raw volume is non-stationary.
Scaler fitted including context (pre-2022) data	training/scaler.py	Mania-era extreme values distort scaler; training features appear out-of-distribution	Fit scaler ONLY on the primary training window (Jan 2022 – present). Assert before fit.

8. Model Architecture & Training Layer

8.1 Model Selection Strategy

✅DOGE Model Build Order — XGBoost First

STEP 1 — Build and validate XGBoost on all post-2022 data (single model, no regime split).
This is the sanity check. If XGBoost cannot beat 53% directional accuracy,
the features do not contain sufficient signal. Fix features before adding LSTM.

STEP 2 — Build regime-specific XGBoost models (one per sub-regime).
Compare each regime model against the global XGBoost. Promote if better.

STEP 3 — Build LSTM on full post-2022 data. LSTM input includes regime_label_encoded.
LSTM captures temporal patterns across regime transitions that XGBoost cannot.

STEP 4 — Build Transformer only if LSTM Sharpe Ratio < 1.0. Otherwise skip.
The Transformer adds significant complexity for marginal DOGE-specific gain.

STEP 5 — Build ensemble meta-learner combining: [xgb_regime_prob, lstm_prob, regime_label].

8.2 Training Data Configuration

□ DOGE Training Window Specification

PRIMARY TRAINING DATASET: DOGEUSDT 1h, January 1 2022 – present
Approximate row count: ~26,000 candles (grows ~720 per month)

WALK-FORWARD PARAMETERS (from config/doge_settings.yaml):

```
training_window_days:     180   # 6 months of 1h candles = ~4,320 rows
validation_window_days:   30    # 1 month = ~720 rows
step_size_days:           7     # Advance 1 week per fold
min_training_rows:        3000  # Minimum rows before first fold begins
```

REGIME-STRATIFIED SAMPLING RULE:
Within each training fold, compute regime distribution.
If any sub-regime has < 5% representation in the fold, apply oversampling
(duplicate regime rows) to bring it to minimum 5% before training.
Log regime distribution for every fold.

PRE-2022 DATA USAGE:
Never included in walk-forward training folds.
Only used to compute: ath_distance, long-term trend context features.
These context features are computed once from pre-2022 data and merged as
static lookup values into the post-2022 feature matrix.

⬜ Training Layer Errors			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**
Training on raw price levels (non-stationary)	training/trainer.py	Model learns absolute price ranges that shift over time	Train on log-returns. Run ADF test on all input features; reject non-stationary features (p > 0.05).
sklearn train_test_split on time series	training/walk_forward.py	Destroys temporal order; catastrophic lookahead bias	BANNED. Only custom walk-forward. Assert: max(train_timestamps) < min(val_timestamps).
Random seed not set for reproducibility	training/trainer.py	Irreproducible results; impossible to debug model differences	Set: random.seed(42), np.random.seed(42), torch.manual_seed(42). Log seed in MLflow.
Regime labels computed after scaler fit	training/scaler.py	Regime boundary timestamps leak via scaler statistics	Compute regime labels BEFORE any scaler fitting. Regime labels are never scaled.
Hyperparameter search on test set	training/hyperopt.py	Overfit to test set; metrics are fiction	Optuna uses only training+validation folds. Test set untouched until final evaluation.
No model archiving before retraining	training/trainer.py	New model overwrites production; no rollback	Archive current model to MLflow with tag 'previous-production' before every retrain.

9. Backtesting & Validation Layer

9.1 Execution Model

✅Realistic Execution Requirements	
Fill Price	Entry and exit execute at NEXT candle's OPEN price. Never fill at signal candle's close.
Taker Fee	0.10% per trade entry and exit. Both legs. Deduct from PnL before any metric computation.
Slippage	Add uniform random slippage between 0.02% and 0.08% per trade.
Position Sizing	Fixed fractional: risk exactly 1% of current equity per trade. Recalculate each trade.

ATR-Adjusted Sizing	[DOGE] In RANGING_LOW_VOL regime: reduce position size to 0.5% of equity.
Max Drawdown Halt	Halt backtest simulation (not just log) if drawdown exceeds 25%. Prevents silent catastrophe.
Short Positions	Enable only if explicitly configured in doge_settings.yaml. Default: long/flat only.
Transaction Log	Log every trade: entry_time, exit_time, regime_at_entry, signal_confidence, pnl_after_fees.

9.2 DOGE-Specific Backtest Rules

DOGE Backtest Requirements

RULE 1 — Run separate backtest reports for each regime. A strategy can be profitable in TRENDING_BULL but catastrophic in RANGING_LOW_VOL. Overall metrics hide this.

RULE 2 — Report performance SEPARATELY for the 2022 bear year and 2023+ recovery period. DOGE behaved very differently in each sub-period. Blended metrics are misleading.

RULE 3 — Backtest must include at least 3 DECOUPLED events to validate that the model handles them correctly (higher threshold, reduced sizing).

RULE 4 — Include a 'no-signal baseline' comparison: how did DOGE perform buy-and-hold over the same test period? The model must outperform buy-and-hold on a risk-adjusted basis (Sharpe and Calmar), not just absolute returns.

RULE 5 — If the backtest test period contains a period where BTC dropped > 20%, report model performance DURING that specific BTC drawdown period separately. DOGE tends to amplify BTC drawdowns — the model must manage this.

DOGE Model Acceptance Gates

Gate	Metric	Minimum Threshold	DOGE Note
G1	Directional Accuracy (out-of-sample)	>= 54%	Measured on post-2022 test set only
G2	Sharpe Ratio (annualized)	>= 1.0	Must also be >= 0.8 PER REGIME (no regime can be catastrophic)
G3	Max Drawdown	<= 20%	DOGE amplifies BTC drawdowns — 20% is a tight but achievable constraint
G4	Calmar Ratio	>= 0.6	Higher than generic threshold due to DOGE volatility profile

G5	Profit Factor	>= 1.3	Gross profit / gross loss across all trades
G6	Win Rate	>= 45%	Lower bound; compensated by avg_win > avg_loss requirement
G7	# Trades	>= 150	More trades needed for DOGE due to higher noise
G8	Outperforms Buy-and-Hold	Sharpe > BH Sharpe	Model must beat passive DOGE holding on risk-adjusted basis
G9	DECOUPLED regime performance	Max drawdown <= 15%	Decoupled events are high-risk; model must reduce exposure correctly

10. Signal Generation & Inference Layer

✅DOGE Inference Pipeline — Complete Step-by-Step

```
Step 1  — FRESHNESS CHECK: assert last candle close_time <= now - interval_ms. Stale = halt.
Step 2  — FEATURE COMPUTATION: run full feature pipeline on last 500 closed candles.
Step 3  — REGIME CLASSIFICATION: run DogeRegimeClassifier on current candle features.
          Log current regime. If regime changed from last candle, log transition event.
Step 4  — FEATURE VALIDATION: assert zero NaN/Inf. Assert feature count == training schema.
Step 5  — SCALING: load scaler saved with current production model. Transform features.
Step 6  — BASE MODEL INFERENCE:
          a. Run LSTM predict_proba(X_sequence)
          b. Run regime-specific XGBoost predict_proba(X_single)
          c. Optionally run Transformer predict_proba(X_sequence)
          Log all individual model outputs.
Step 7  — REGIME-ADJUSTED THRESHOLD: look up confidence_threshold from regime_config.yaml
          Default thresholds by regime:
            TRENDING_BULL / TRENDING_BEAR:  0.62
            RANGING_HIGH_VOL:               0.65
            RANGING_LOW_VOL:                0.70  (reduce noise in choppy market)
            DECOUPLED:                      0.72  (highest risk state)
Step 8  — ENSEMBLE: feed [lstm_prob, xgb_prob, regime_encoded] to meta-learner.
Step 9  — SIGNAL DECISION:
          if ensemble_prob >= threshold:        emit BUY
          elif ensemble_prob <= 1-threshold:    emit SELL (if shorts enabled)
          else:                                 emit HOLD
Step 10 — RISK FILTER: apply all risk rules from Section 11.
          If any risk rule fires: suppress signal and log reason.
Step 11 — LOG full prediction record:
```

```
        {timestamp, symbol, regime, lstm_prob, xgb_prob, ensemble_prob,
         threshold_used, signal, confidence, feature_hash, model_version}
Step 12 — EMIT signal to downstream consumer.
```

⬜ Inference Errors			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**
Feature schema mismatch at inference	inference/engine.py	Silent wrong predictions or crash	Save feature_columns.json with model. Assert exact match at inference time.
Wrong regime threshold used	inference/signal.py	Signal emitted at wrong confidence level for current regime	Load threshold from regime_config.yaml using current regime key. Assert threshold in [0.55, 0.80].
Predicting on incomplete (unclosed) candle	inference/engine.py	Features based on partial data; all indicators wrong	Only trigger on is_candle_closed == True from WebSocket. Never trigger on open candle.
HTF features not updated at HTF boundary	inference/engine.py	4h feature unchanged for 4 hours; stale context	At each 1h candle, check if a new 4h candle has closed. If yes, re-fetch and recompute HTF features.
Funding rate stale at inference	features/funding_features.py	Feature uses 8h-old funding rate as if current	Forward-fill funding rate to current 1h candle. Log age of last funding rate update.

11. Deployment & Monitoring Layer

11.1 DOGE-Specific Monitoring Metrics

Live Monitoring — DOGE Edition			
Metric	**Collection**	**Alert Threshold**	**DOGE-Specific Note**
Current regime label	Custom gauge (Prometheus)	Regime change within 2 candles → alert	Log every regime transition; track regime duration histogram
doge_btc_corr_24h	Custom gauge	< 0.30 → DECOUPLED alert (critical)	Decoupled events require immediate threshold adjustment
Volume ratio (current)	Custom gauge	Spike > 3.0 → log; > 6.0 → high alert	Volume spike is a leading signal — log immediately

Funding rate z-score	Custom gauge	| z-score | > 2.5 → extreme flag alert	Extremes precede reversals; requires human awareness
BTC correlation drop rate	Custom gauge	corr drops > 0.15 in 2 candles → alert	Fast correlation drops indicate impending decoupling
Inference latency (p99)	Prometheus histogram	> 500ms → alert	Multi-symbol feature computation is heavier than single-symbol
Signal frequency (per 24h)	Counter	< 1 signal/day OR > 48 signals/day → alert	Too few = model too conservative; too many = noise-driven
Rolling 48h accuracy	Custom gauge	< 50% → trigger retraining workflow	DOGE regime shifts make accuracy more volatile than BTC
Equity drawdown	Custom gauge	> 10% → halt + alert; > 20% → emergency stop	DOGE drawdowns are fast; 10% drawdown trigger is appropriate
Data freshness	Custom gauge	Last candle > 2 * interval → alert	Multiple sources (DOGE, BTC, DOGEBTC) all must be fresh

12. Complete Error & Failure Catalogue

12.1 DOGE Sentiment-Driven Failure Modes

DOGE-Specific Failure Modes			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**
Sudden BTC crash propagating to DOGE (amplified beta)	inference/signal.py	BTC drops 10% → DOGE drops 15-20%; model was long DOGE	Add BTC hourly return as emergency halt input. If btc_1h_return < -4%, suppress all DOGE BUY signals.
DOGE mania recurrence detection failure	regimes/classifier.py	DOGE enters new mania; model trained on current regime fails catastrophically	Monitor 7d return. If > +80%, trigger MANIA_ALERT regime. Reduce position to 25%. Increase threshold to 0.80.
Funding rate spike precedes reversal but signal is still BUY	inference/signal.py	Enters long position into crowded long → sharp reversal	Add funding override rule: if funding_extreme_long == 1, suppress BUY signals regardless of model output.

Round number rejection mistaken for breakout	features/doge_specific.py	Price approaches $0.20; model predicts breakout; it reverses	at_round_number_flag must reduce position size by 30% on BUY signals near major round levels.
Volume spike in bear regime treated same as bull regime	features/doge_specific.py	Bear market volume spike is sell climax, not buy opportunity	Volume spike features must be regime-conditioned. Log volume_spike_flag AND current_regime together.

12.2 Statistical & Mathematical Errors

Statistical Errors			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**
Lookahead bias via same-candle target	features/pipeline.py	Fantasy backtest accuracy; zero live edge	Target: close.pct_change().shift(-1) > 0. Write unit test. Run before every training job.
Scaler fitted on full dataset	training/scaler.py	Future statistics leak into normalization	Re-fit scaler on each walk-forward training fold only. Assert no future timestamps in fit data.
Non-stationary input features	training/trainer.py	Model breaks outside trained price range	Run ADF test on all features. Reject series with p > 0.05. Use returns not levels.
Class imbalance ignored	training/trainer.py	Model predicts majority class; looks accurate but has no edge	Compute class_weight. Apply scale_pos_weight (XGBoost) or class_weight='balanced' (sklearn).
Multiple comparison bias in feature selection	features/pipeline.py	False signal discovery from testing 50+ features	Apply Bonferroni correction. Only include features with adjusted_p < 0.05/n_features.
Autocorrelated residuals ignored	evaluation/metrics.py	Standard errors underestimated; metrics look better than reality	Run Durbin-Watson on residuals. If DW < 1.5 or > 2.5, model is misspecified.

12.3 Infrastructure Errors

Infrastructure & Operational Errors			
Error / Mistake	**Where It Occurs**	**Consequence**	**How to Detect & Fix**

Multi-symbol WebSocket gap: BTC drops but DOGE stream alive	ws_client.py	BTC features go stale; doge_btc_corr computed on stale BTC data	Watchdog must monitor ALL symbol streams. If BTC stream stale, suspend DOGE inference.
Funding rate API returns empty (futures maintenance)	ingestion/futures_client.py	funding_features return NaN → feature matrix fails validation	Handle empty funding rate response: forward-fill from last known value. Log maintenance event.
TimescaleDB hypertable query slow on large date range	processing/storage.py	Bootstrap takes hours; timeout errors	Add time-based index. Use chunk_time_interval='1 month' for hypertable. Query in date-range pages.
Docker memory limit exceeded during LSTM training	training/trainer.py	OOM kill after hours of training; no checkpoint	Set batch_size=32 if memory constrained. Checkpoint model weights every 10 epochs.
MLflow server down during training run	training/trainer.py	Metrics not logged; model lost	Wrap all mlflow calls in try/except. Fall back to local JSON logging. Never let MLflow crash training.

13. Anti-Pattern Registry

□ Anti-Pattern #1 — Optimizing Backtest Until It Looks Good

Symptom: Agent repeatedly tunes parameters because the backtest metrics look poor.

Problem: Each optimization iteration fits the model more to historical noise.

Hyperparameter search must only touch training+validation folds.

The test set is examined ONCE after all optimization is complete.

DOGE-Specific Risk: The 2021 mania creates extreme outlier events. If the agent tunes on data that includes the mania, it will produce a model that 'learned' to predict the once-in-a-cycle event, which will never happen the same way again.

□ Anti-Pattern #2 — Treating DOGE Like BTC

Symptom: Agent uses a generic crypto feature set without DOGE-specific features.

Problem: DOGE's sentiment-driven, high-beta, retail-dominated nature makes it respond to different signals than BTC. A BTC-tuned model on DOGE data will underperform because it ignores the 12 mandatory DOGE features.

Rule: All 12 DOGE-specific features must be present in every training run.

Any training run missing even one DOGE-specific feature is invalid.

Anti-Pattern #3 — Ignoring Regime During Inference

Symptom: Same model weights and same confidence threshold used for all market states.
Problem: A threshold tuned for trending markets will over-trade in ranging markets.
A threshold tuned for ranging markets will miss trending opportunities.
Rule: Regime classification must run BEFORE inference on every candle.
Confidence threshold is always loaded from regime_config.yaml, not hardcoded.

Anti-Pattern #4 — Long Signal During Extreme Funding Rate

Symptom: Model emits BUY signal. Funding rate is +0.15% per 8h (extreme long crowding).
Problem: Extreme positive funding means leveraged longs are dominant. When they unwind, DOGE drops sharply and suddenly. The model's BUY signal becomes an entry into a liquidation cascade.
Rule: funding_extreme_long == 1 SUPPRESSES all BUY signals. This is a hard override.
The model signal does not matter when funding is extreme.

Anti-Pattern #5 — Not Testing Regime Performance Separately

Symptom: Agent reports a single Sharpe ratio and declares model acceptable.
Problem: DOGE regimes have very different characteristics. A model can be excellent in TRENDING_BULL and catastrophic in RANGING_LOW_VOL. Blended metrics hide per-regime disaster.
Rule: ALWAYS report Sharpe, max drawdown, and accuracy PER REGIME.
No model is accepted until all regime-specific metrics pass their gates.

Anti-Pattern #6 — Including Pre-2022 Rows in Training Folds

Symptom: Agent includes all available DOGE history (back to 2019) in training.
Problem: The statistical properties of 2019-2021 DOGE are fundamentally different. The mania period (2021) will teach the model patterns that cannot recur in their original form. Including these rows degrades model quality.
Rule: Primary training dataset starts January 1 2022. Earlier data is context only.
Pre-2022 data is never in a training fold. This is enforced by era partitioning.

14. Self-Correction & Recovery Protocols

14.1 Regime Transition Protocol

Protocol: Regime Change Detected

TRIGGER: DogeRegimeClassifier returns a different regime label than the previous candle.

STEP 1 — LOG: regime_change_at, from_regime, to_regime, trigger_features (corr, bb_width, etc.)
STEP 2 — SWAP THRESHOLD: immediately load new confidence_threshold from regime_config.yaml.
STEP 3 — If transitioning TO DECOUPLED:
  a. Set confidence_threshold = 0.72
  b. Send DECOUPLED_ALERT to monitoring system
  c. Reduce position size to 50% of base for all new signals
STEP 4 — If transitioning FROM DECOUPLED:
  a. Wait 3 candles before resuming normal threshold (regime stabilization)
  b. Log re-coupling event with duration of decoupled period
STEP 5 — Do NOT retrain on regime transition alone. Retrain only if accuracy drops.
STEP 6 — Tag all signals with the regime active at the time of signal generation.

14.2 Data Gap Recovery

✅Protocol: Gap Detected in DOGE or BTC Data

TRIGGER: Validator detects timestamp gap > interval_ms in any primary data source.

STEP 1 — IDENTIFY which symbol has the gap (DOGE, BTC, or DOGEBTC).
STEP 2 — QUANTIFY: missing_candles = gap_ms / interval_ms.
STEP 3 — ASSESS:
  missing_candles <= 3: backfill via REST API silently.
  missing_candles 4-50: backfill + tag as backfilled=True.
  missing_candles > 50: HALT. Log critical. Full re-validation required.
STEP 4 — CROSS-ASSET CHECK: if gap is in BTC data, suspend DOGE inference until BTC gap is recovered. doge_btc_corr feature is invalid without BTC data.
STEP 5 — VERIFY: re-run validator + aligner. Assert all symbols in sync.
STEP 6 — RESUME inference only after validator confirms alignment.

14.3 Accuracy Degradation Protocol

✅Protocol: Rolling 48h Accuracy < 50%

TRIGGER: rolling_48h_accuracy < 0.50 in monitoring metrics.

STEP 1 — VERIFY it is not a data gap or stale data artifact. Check feature freshness.
STEP 2 — DIAGNOSE:
  a. Has a regime transition occurred in the last 48h?
  b. Has BTC experienced a major move (>8% in 24h)?
  c. Is the DECOUPLED regime active?
  d. Has feature distribution drifted > 2 std from training baseline?

```
STEP 3 — RESPOND:
        Regime transition recently:  increase threshold +0.05 for 24h, monitor.
        BTC major move:              add BTC return as emergency override feature.
        DECOUPLED:                   already handled by regime protocol above.
        Feature drift detected:      switch to HOLD mode, trigger retraining.
        No clear cause:              switch to HOLD mode, full diagnostic retrain.
STEP 4 — New model must pass all Section 9.2 gates before promotion.
STEP 5 — Shadow mode 48h minimum before replacing production model.
```

14.4 Drawdown Recovery Protocol

⚠ Protocol: Equity Drawdown Breach

```
TRIGGER: Live equity drawdown exceeds 10% from peak.

STEP 1 — IMMEDIATELY switch to HOLD mode. No new signals.
STEP 2 — Preserve ALL logs: signal log, feature log, regime log, funding rate log.
STEP 3 — DIAGNOSE: was the drawdown caused by:
        a. A BTC systemic crash (DOGE amplified the move)?
        b. A DOGE-specific event (delisting threat, exchange issue, sentiment dump)?
        c. Model error (wrong regime, wrong threshold, funding override not triggered)?
STEP 4 — If BTC systemic: resume with reduced position sizing (0.5% risk per trade).
        If DOGE-specific event: wait for regime re-stabilization (minimum 5 days).
        If model error: fix the specific error, retrain, validate, shadow mode.
STEP 5 — Drawdown > 20%: FULL STOP. No automated decisions. Human review required.
```

15. Quality Gates & Acceptance Criteria

All nine quality gates must pass before the corresponding pipeline stage advances. A failing gate halts the pipeline. The gate outcome must be logged to MLflow.

Quality Gate Registry — DOGE Edition

Gate	Stage	Criteria	Action if Fail
QG-01	Post-Bootstrap	Row counts match expected ± 2 for ALL 5 sources. Zero gaps. Aligner confirms identical timestamps across DOGE, BTC, DOGEBTC.	Re-run bootstrap for missing ranges. Debug aligner for symbol-specific gaps.
QG-02	Post-Processing	Zero NaN/Inf in all OHLCV. Schema matches pandera contract. Era	Identify source rows. Fix ingestion. Re-process from raw.

		partition correct. Funding rates 8h-interval validated.	
QG-03	Post-Features	Zero NaN/Inf in feature matrix. All 12 DOGE-specific features present. Regime labels assigned to every row. No constant features.	Debug feature pipeline. Ensure dropna ran. Check DOGE-specific modules.
QG-04	Regime Validation	Regime distribution logged for full training set. No regime with < 2% representation. Regime transition test passed.	Adjust regime boundaries in regime_config.yaml. Re-run classifier.
QG-05	Pre-Training	Scaler fitted on training fold only. Era = 'context' rows excluded. Seed set. Class weights computed. XGBoost sanity check > 53%.	If XGBoost < 53%: features insufficient. Fix features before training LSTM.
QG-06	Post-Backtest	ALL Section 9.2 gates pass. Per-regime Sharpe all >= 0.8. DECOUPLED regime drawdown <= 15%. Outperforms buy-and-hold on Calmar.	Do NOT deploy. Diagnose per-regime failure. Tune or re-engineer features.
QG-07	Pre-Deployment	All tests pass (zero failures). Coverage >= 80%. Docker build clean. Shadow mode 48h complete. Scaler + feature_columns.json packaged.	Fix failing tests. Extend shadow period. Resolve all build errors.
QG-08	Post-Deployment	First 100 live signals logged. Per-signal regime logged. Latency < 500ms (p99). All monitoring metrics visible in Grafana.	If accuracy < 50% after 100 signals: rollback immediately.
QG-09	Retraining Gate	New model Sharpe > current production on last 30 days. Per-regime metrics all pass. Shadow accuracy > production accuracy.	Do not promote. Investigate degradation cause. Keep current production active.

16. Agent Execution Checklist

Work through every item in sequence. No step may be skipped. Each item must be explicitly completed and logged before proceeding to the next.

Phase 1 — Project Initialization

1. Create full directory structure from Section 1 — including all [DOGE] directories
2. Initialize git. Add .gitignore: secrets.env, .venv/, data/raw/, mlruns/, data/features/
3. Create virtual environment (Python 3.11+). Install all pinned dependencies

4. Verify ta-lib C library installed at OS level before pip install ta-lib
5. Create config/settings.yaml, config/doge_settings.yaml, config/regime_config.yaml
6. Write pandera schema contracts for: DOGEUSDT OHLCV, DOGEBTC OHLCV, funding rates
7. Write placeholder test files for all src/ modules. Confirm pytest runs without errors

Phase 2 — Multi-Symbol Data Bootstrap

8. Implement BinanceRESTClient with rate limit tracking, retry logic, weight header parsing
9. Implement BinanceFuturesClient for funding rate endpoint
10. Run DOGEUSDT 1h bootstrap (Jul 2019 – present). Save checkpoint every 5,000 rows
11. Run BTCUSDT 1h bootstrap concurrently. Verify both complete before proceeding
12. Run DOGEBTC 1h bootstrap. Verify row count matches DOGEUSDT
13. Run 4h and 1d bootstraps for DOGEUSDT
14. Run funding rate bootstrap (Oct 2020 – present). Validate 8h intervals
15. Run aligner. Assert identical timestamp index across DOGE, BTC, DOGEBTC. QG-01 must pass
16. Run validator on all 5 sources. QG-02 must pass

Phase 3 — Regime Classification

17. Implement DogeRegimeClassifier with all 5 sub-regimes
18. Write unit tests for regime classifier: each regime correctly identified from synthetic data
19. Run label_regimes.py on full post-2022 dataset. Save to data/regimes/
20. Log regime distribution. Verify all 5 regimes present. QG-04 must pass
21. Visualise regime timeline in notebooks/regime_analysis.ipynb (exploration only)

Phase 4 — Feature Engineering

22. Implement all standard features: price indicators, volume indicators, lag features
23. Implement all 12 DOGE-specific features in doge_specific.py and funding_features.py
24. Implement HTF features: 4h and 1d derived features with correct lookahead protection
25. Write unit test for EVERY feature — especially: lag_1_close[t] == close[t-1]
26. Write BTC correlation unit test: assert result uses log-returns, not raw prices
27. Write HTF boundary unit test: assert 4h feature at 15:00 uses candle closed at 12:00
28. Run full feature pipeline on post-2022 dataset. Assert zero NaN/Inf. QG-03 must pass
29. Print feature correlation matrix. Drop features with |corr| > 0.98

Phase 5 — Model Training

30. Implement BaseModel abstract class. All models must inherit from it

31. Implement walk-forward CV engine. Verify scaler re-fitted per fold, pre-2022 excluded
32. FIRST: Train single global XGBoost. Verify directional accuracy > 53% (QG-05 sanity check)
33. If XGBoost < 53%: STOP. Debug feature pipeline before any further model work
34. Train regime-specific XGBoost models (one per sub-regime). Log each to MLflow
35. Train LSTM with regime_label_encoded as input feature
36. Train ensemble meta-learner on base model outputs + regime label
37. Run hyperparameter search (Optuna) on train+val folds only. Never touch test set
38. Archive all models to MLflow with: feature_columns.json, scaler.pkl, regime_config.yaml

Phase 6 — Backtesting

39. Implement backtesting engine: next-candle-open fill, 0.1% fees, slippage, ATR position sizing
40. Run backtest on held-out test set (never seen in training or tuning)
41. Generate per-regime performance reports (mandatory, not optional)
42. Generate buy-and-hold comparison report
43. QG-06 must pass: all Section 9.2 gates including per-regime Sharpe >= 0.8

Phase 7 — Deployment

44. Implement inference engine with: regime classification, HTF update logic, funding override
45. Test all risk overrides: funding_extreme_long suppresses BUY, round_number reduces sizing
46. Build Docker image. Verify health check endpoint
47. Deploy in shadow mode. Run minimum 48 hours
48. QG-07 and QG-08 must pass
49. Configure Grafana: all Section 11.1 metrics visible including regime label and doge_btc_corr
50. Configure alerting: DECOUPLED alert, drawdown halt, accuracy degradation, data freshness
51. Test rollback to previous MLflow model version. Verify it completes in under 5 minutes

Phase 8 — Ongoing Operations

52. Schedule weekly retraining. Verify shadow mode activates before promotion
53. Schedule monthly: review regime config thresholds, update round number levels if price moved
54. After first 100 live signals: review per-regime signal distribution. Is any regime over-represented?
55. After 30 live trading days: run full backtest on recent live data vs live signals. Compare.
56. Quarterly: re-evaluate whether pre-2022 context features remain relevant

17. Self-Teaching Reinforcement Learning System

This section defines the complete architecture for the model's self-teaching capability. The model generates predictions, stores them, verifies them against real outcomes, and updates its weights based on a structured reward/punishment function — improving continuously after deployment.

✓Core RL Loop — Conceptual Overview

At every candle close, the model performs two jobs simultaneously:

JOB 1 — PREDICT: Generate price predictions for 4 future time horizons.
Store each prediction in the Prediction Store with full metadata.

JOB 2 — VERIFY: Check the Prediction Store for predictions whose target time has now been reached. For each matured prediction:

a. Fetch the actual DOGE close price at the target timestamp
b. Compute the reward or punishment score
c. Write the outcome back to the Prediction Store
d. Add the (features, prediction, reward) tuple to the Replay Buffer

Periodically, when triggered by schedule or buffer fill level, the model runs a self-training update using the Replay Buffer — updating its weights to increase cumulative future reward. This cycle repeats indefinitely.

The model is always predicting, always verifying, always learning.

17.1 Prediction Horizons

The model generates predictions across four time horizons at every candle close. Each horizon serves a different purpose and carries different reward and punishment weights.

Prediction Horizon Specification

Horizon	Label	Target	Purpose	Reward Weight	Punishment Weight
4 hours	SHORT	T + 4 candles	Tactical momentum; fastest feedback loop; highest learning frequency	1.0x	1.5x
24 hours	MEDIUM	T + 24 candles	Swing position; daily trend; regime-aligned signals	1.5x	2.0x

7 days	LONG	T + 168 candles	Positional trend; macro cycle; regime transition detection	2.0x	2.5x
30 days	MACRO	T + 720 candles	Long-term directional bias; sparse but high-value signal	1.5x	1.0x

□ DOGE Horizon Calibration Notes

SHORT (4h): Most learnable for DOGE. Sentiment signals and volume spikes decay within hours. Fastest feedback loop. Drives the most weight updates. Highest trading value.

MEDIUM (24h): Captures daily trend momentum. BTC correlation most stable at this range. Funding rate signals are particularly predictive at 24h horizon.

LONG (7d): Captures regime-level directional moves. Regime label at prediction time is the most important context variable for evaluating 7d outcome.

MACRO (30d): DOGE 30d behaviour is heavily influenced by unpredictable events (halvings, news, celebrity activity). Lower punishment weight reflects this noise. Model learns directional bias only, not precision.

17.2 Prediction Store Schema

Every prediction is written to the Prediction Store immediately after generation. The store is append-only — records are never deleted. Only outcome fields are updated when a horizon matures.

✅Prediction Store — Full Database Schema

Table: doge_predictions (TimescaleDB hypertable, partitioned by predicted_at_ts)

```
prediction_id       UUID PRIMARY KEY DEFAULT gen_random_uuid()
created_at          TIMESTAMPTZ NOT NULL DEFAULT now()

-- Context at prediction time (IMMUTABLE after insert)
predicted_at_ts     BIGINT NOT NULL    -- candle open_time at prediction (UTC ms)
symbol              VARCHAR(20) NOT NULL DEFAULT 'DOGEUSDT'
price_at_prediction NUMERIC(18,8) NOT NULL  -- close price at candle T
horizon_label       VARCHAR(10) NOT NULL    -- SHORT | MEDIUM | LONG | MACRO
horizon_candles     INT NOT NULL            -- 4 | 24 | 168 | 720
target_ts           BIGINT NOT NULL         -- predicted_at_ts + horizon_candles * 3600000

-- Prediction outputs (IMMUTABLE after insert)
```

```
predicted_price      NUMERIC(18,8) NOT NULL
predicted_direction  SMALLINT NOT NULL        -- 1=UP  -1=DOWN  0=FLAT
confidence_score     NUMERIC(5,4) NOT NULL    -- [0.5, 1.0]

-- Model metadata (IMMUTABLE after insert)
model_version        VARCHAR(64) NOT NULL     -- MLflow run_id
regime_at_prediction VARCHAR(20) NOT NULL
curriculum_stage     SMALLINT NOT NULL        -- 1 | 2 | 3 | 4
feature_hash         VARCHAR(64) NOT NULL     -- SHA256 of scaled feature vector

-- Outcome fields (NULL until verified)
actual_price         NUMERIC(18,8)
actual_direction     SMALLINT
price_error_pct      NUMERIC(10,6)
direction_correct    BOOLEAN
reward_score         NUMERIC(10,6)
direction_score      NUMERIC(6,4)
magnitude_score      NUMERIC(6,4)
calibration_score    NUMERIC(6,4)
regime_at_target     VARCHAR(20)
verification_failed  BOOLEAN DEFAULT FALSE
fail_reason          VARCHAR(200)
verified_at          TIMESTAMPTZ

-- Required indexes
CREATE INDEX idx_target_unverified ON doge_predictions(target_ts)
  WHERE verified_at IS NULL AND verification_failed = FALSE;
CREATE INDEX idx_horizon_created   ON doge_predictions(horizon_label, created_at);
CREATE INDEX idx_model_version     ON doge_predictions(model_version);
CREATE INDEX idx_regime_prediction ON doge_predictions(regime_at_prediction);
```

Prediction Store — Immutability Rules

- NEVER delete rows from doge_predictions. It is a permanent audit trail.
- NEVER update any field in the first block (predicted_at_ts through feature_hash). These fields are written once at insertion and are immutable.
- NEVER run the verifier on a prediction whose target_ts > current UTC candle time.
- NEVER mark a prediction verified if actual_price came from a gap or interpolated candle. Set verification_failed=TRUE and fail_reason='interpolated_candle' instead.
- Back up the Prediction Store before every self-training run.

17.3 The Reward Function

The reward function is the most critical component of the entire RL system. It determines what behaviour the model learns to optimize. A poorly designed function produces a model that games the metric rather than predicting accurately.

✅Reward Function — Three-Component Architecture

```
final_reward = direction_score  ×  magnitude_score  ×  abs(calibration_score)

If direction_score is negative, the final reward is negative (punishment mode).
If any component is zero, the total reward is zero.

──────────────────────────────────────────────────────────────────────────────
COMPONENT 1 — Direction Score  (range: −1.0 to +1.0)
──────────────────────────────────────────────────────────────────────────────
  predicted_direction == actual_direction  →  +1.0
  predicted_direction != actual_direction  →  −1.0
  predicted_direction == 0 (FLAT)          →  +0.1  (hedging strongly discouraged)

Flat predictions earn almost no reward even when technically correct.
The model must learn to take a directional stance.

──────────────────────────────────────────────────────────────────────────────
COMPONENT 2 — Magnitude Score  (range: 0.0 to 1.0)
──────────────────────────────────────────────────────────────────────────────
  error_pct = |predicted_price − actual_price| / actual_price × 100
  magnitude_score = exp(−decay_constant × error_pct)

  Decay constants by horizon (from rl_config.yaml):
    SHORT:  0.035  →  error_pct 5% gives magnitude_score 0.84
    MEDIUM: 0.025  →  error_pct 5% gives magnitude_score 0.88
    LONG:   0.015  →  error_pct 5% gives magnitude_score 0.93
    MACRO:  0.010  →  error_pct 5% gives magnitude_score 0.95

  In DECOUPLED regime: decay_constant × 0.5 (more tolerant of error)

──────────────────────────────────────────────────────────────────────────────
COMPONENT 3 — Confidence Calibration Score  (range: −3.0 to +2.0)
──────────────────────────────────────────────────────────────────────────────
  When prediction is CORRECT:
```

```
    calibration_score = 1.0 + (confidence_score − 0.5) × 2.0
    confidence=0.80 correct  →  calibration = +1.60  (amplified reward)
    confidence=0.55 correct  →  calibration = +1.10  (modest reward)

  When prediction is WRONG:
    calibration_score = −(1.0 + (confidence_score − 0.5) × 4.0)
    confidence=0.80 wrong    →  calibration = −2.20  (severe punishment)
    confidence=0.55 wrong    →  calibration = −1.20  (mild punishment)

This asymmetry teaches: be confident only when you have genuine edge.
Overconfident wrong predictions receive disproportionately high punishment.
```

✅compute_reward() — Full Python Implementation

```
import numpy as np
from dataclasses import dataclass

@dataclass
class RewardResult:
    reward_score:      float
    direction_score:   float
    magnitude_score:   float
    calibration_score: float
    error_pct:         float
    direction_correct: bool

def compute_reward(
    predicted_price:      float,
    actual_price:         float,
    price_at_prediction:  float,  # close at T (NOT predicted_price)
    predicted_direction:  int,    # 1 | -1 | 0
    confidence_score:     float,  # [0.5, 1.0]
    horizon_label:        str,
    regime_at_prediction: str,
    config:               dict,   # from rl_config.yaml
) -> RewardResult:

    # Actual direction is always vs price_at_prediction (price when predicted)
    actual_direction = 1 if actual_price > price_at_prediction else -1
    error_pct = abs(predicted_price - actual_price) / actual_price * 100

    # ── Direction ──────────────────────────────────────────────────────
    if predicted_direction == 0:
        direction_score = 0.1
```

```
    elif predicted_direction == actual_direction:
        direction_score = 1.0
    else:
        direction_score = -1.0

    # ── Magnitude ──────────────────────────────────────────────────────
    decay = config['decay_constants'][horizon_label]
    if regime_at_prediction == 'DECOUPLED':
        decay *= config['decoupled_decay_multiplier']
    magnitude_score = float(np.exp(-decay * error_pct))

    # ── Confidence Calibration ─────────────────────────────────────────
    is_correct = direction_score > 0
    conf_excess = confidence_score - 0.5
    if is_correct:
        calibration_score = 1.0 + conf_excess * 2.0
    else:
        calibration_score = -(1.0 + conf_excess * 4.0)

    # ── Final Score with horizon weights ───────────────────────────────
    hw = config['horizons'][horizon_label.lower()]
    raw = direction_score * magnitude_score * abs(calibration_score)
    if raw >= 0:
        final_score = raw * hw['reward_weight']
    else:
        final_score = raw * hw['punish_weight']

    return RewardResult(
        reward_score      = round(final_score, 6),
        direction_score   = direction_score,
        magnitude_score   = round(magnitude_score, 4),
        calibration_score = round(calibration_score, 4),
        error_pct         = round(error_pct, 4),
        direction_correct = is_correct,
    )
```

17.4 Reward Score Reference Table

Reward Score Examples — Key Scenarios					
Scenario	**Direction**	**Error %**	**Confidence**	**Reward Score**	**Lesson**

Perfect SHORT prediction	Correct	0.3%	0.90	+ 1.54	Best possible outcome — high confidence, high accuracy
Good MEDIUM, moderate error	Correct	3.0%	0.75	+ 1.73	Correct direction, acceptable error, confident
Correct but barely confident	Correct	2.0%	0.55	+ 0.56	Model was right but barely tried — weak reward
Wrong, low confidence	Wrong	4.0%	0.55	− 1.44	Model was appropriately uncertain — mild punishment
Wrong, high confidence SHORT	Wrong	3.0%	0.85	− 3.30	Overconfident and wrong — severe punishment (worst case)
Wrong, high confidence LONG	Wrong	5.0%	0.80	− 6.60	LONG carries 2.5x punish weight — most severe
FLAT prediction (hedging)	Flat	1.0%	0.70	+ 0.09	Model is hedging — near-zero reward even when right
DECOUPLED regime, wrong	Wrong	6.0%	0.70	− 1.80	Decay halved in DECOUPLED — punishment reduced

17.5 Replay Buffer & Self-Training Loop

✓Replay Buffer Specification

Storage: TimescaleDB table 'doge_replay_buffer' (separate from predictions)
Max size: 10,000 records per horizon buffer (four buffers total)
Eviction: Oldest records evicted when buffer is full (FIFO within priority class)

Prioritised sampling: records with |reward_score| > 2.0 are sampled 3x more frequently during self-training. High-magnitude outcomes — both highly correct and severely wrong — contain the most informative learning signal.

Regime-stratified: maintain MINIMUM 100 records per regime per buffer regardless of age. This prevents rare regimes (DECOUPLED, RANGING_LOW_VOL) from being evicted before the model has learned from them.

Contents of each buffer record:
- feature_vector_scaled — exact input array used at prediction time (float32)
- horizon_label — SHORT | MEDIUM | LONG | MACRO
- predicted_price — model output

```
actual_price          — verified outcome
reward_score          — computed reward
regime_at_prediction  — regime context for regime-conditional sampling
model_version         — which model generated the prediction
```

Self-Training Trigger Conditions

```
Self-training runs when ANY of the following conditions are met:

TRIGGER 1 — SCHEDULED: Every Sunday 02:00 UTC
TRIGGER 2 — BUFFER FULL: Any horizon buffer reaches 80% capacity (8,000 records)
TRIGGER 3 — REWARD DECLINE: Rolling 7-day mean reward_score < 0.0
            (model earning net punishment — learning is reversing)
TRIGGER 4 — REGIME ACCUMULATION: 50+ new verified predictions in a new regime
            since last training (fast adaptation to regime change)

Minimum cooldown between self-training runs: 48 hours
  (Prevents oscillation from too-frequent weight updates)

Self-Training Process:
  1. Lock Replay Buffer for reading (no concurrent writes)
  2. Sample batch from each horizon buffer (prioritised, regime-stratified)
  3. XGBoost: re-fit using replay samples weighted by |reward_score|
  4. LSTM: gradient update steps using reward as reinforcement signal
     (positive reward = reinforce activations; negative = penalize via gradient reversal)
  5. Meta-learner: re-fit on recent base model outputs + reward outcomes
  6. Validate new weights on held-out last-30-days verified predictions
  7. If new model rolling reward_score > current production: promote
  8. If not: retain current weights, log comparison, extend buffer collection
```

17.6 Curriculum Learning Protocol

The model earns the right to predict at longer horizons by first demonstrating competence at shorter horizons. This prevents the model from being destabilized by long-horizon noise before it has mastered short-term patterns.

Curriculum Stages

```
STAGE 1 — SHORT HORIZON ONLY (4h predictions)
  Begins: Day 1 of live deployment
  Minimum verified predictions before advancement check: 1,000
  Minimum days in stage before advancement: 14
  Advancement criteria:
    a. Rolling 14-day directional accuracy on SHORT > 54%
```

```
    b. Rolling 14-day mean reward_score on SHORT > 0.30
  Max wait: 60 days — if not met, trigger full diagnostic review

STAGE 2 — ADD MEDIUM (4h + 24h predictions)
  Advancement criteria:
    a. Rolling 21-day accuracy on MEDIUM > 53%
    b. Rolling 21-day mean reward_score on MEDIUM > 0.20

STAGE 3 — ADD LONG (4h + 24h + 7d predictions)
  Advancement criteria:
    a. Rolling 28-day accuracy on LONG > 52%
    b. Max drawdown from LONG-guided signals < 15%

STAGE 4 — ADD MACRO (all four horizons active)
  No advancement criteria — MACRO is the final stage.
  Monitor MACRO reward distribution separately — lower accuracy expectations.

Rationale: 4h feedback arrives in hours. 30d feedback takes a month.
Starting with 4h means the model gets thousands of learning cycles before
it must grapple with the noise of 30d DOGE prediction.
```

17.7 Verifier Implementation

✅PredictionVerifier — Required Logic

```
class PredictionVerifier:
    def run(self, current_candle_ts: int) -> int:
        '''Runs at every candle close. Returns count verified.'''

        # Step 1: Find matured unverified predictions
        pending = self.store.get_matured_unverified(as_of_ts=current_candle_ts)

        verified_count = 0
        for pred in pending:

            # Step 2: Fetch actual candle at target_ts
            candle = self.ohlcv.get_candle_by_open_time(pred.target_ts)

            # Step 3: Reject if candle unavailable or interpolated
            if candle is None or candle.is_interpolated:
                self.store.mark_failed(pred.prediction_id,
                    reason='candle_unavailable_or_interpolated')
```

```
            continue

        # Step 4: Compute actual direction vs price AT PREDICTION TIME
        actual_dir = 1 if candle.close > pred.price_at_prediction else -1

        # Step 5: Get regime at target time
        regime_target = self.regime_clf.get_at(pred.target_ts)

        # Step 6: Compute reward
        result = compute_reward(
            predicted_price      = pred.predicted_price,
            actual_price         = candle.close,
            price_at_prediction  = pred.price_at_prediction,
            predicted_direction  = pred.predicted_direction,
            confidence_score     = pred.confidence_score,
            horizon_label        = pred.horizon_label,
            regime_at_prediction = pred.regime_at_prediction,
            config               = self.config,
        )

        # Step 7: Write outcome to Prediction Store
        self.store.write_outcome(pred.prediction_id, result, regime_target)

        # Step 8: Push to Replay Buffer
        self.replay_buffer.push(pred, result)

        logger.info(
            f'Verified {pred.horizon_label} | reward={result.reward_score:.4f}'
            f' | correct={result.direction_correct} | err={result.error_pct:.2f}%'
        )
        verified_count += 1

    return verified_count
```

17.8 Directory & Config Additions

✅RL System — New Directories & Files

```
doge_predictor/
├── src/rl/
│   ├── __init__.py
│   ├── predictor.py       # Multi-horizon prediction generator
```

```
│   ├── verifier.py        # Matured prediction verification job
│   ├── reward.py          # compute_reward() — core reward function
│   ├── replay_buffer.py   # Buffer management + prioritised sampling
│   ├── rl_trainer.py      # Self-training weight update loop
│   ├── curriculum.py      # Stage manager + advancement checks
│   └── rl_monitor.py      # RL Prometheus metrics
├── data/
│   ├── predictions/       # Prediction Store nightly backup snapshots
│   └── replay_buffers/    # Serialised buffer checkpoints
├── config/
│   └── rl_config.yaml     # All RL hyperparameters (see below)
├── tests/unit/
│   ├── test_reward.py     # Unit tests for all 8 reward scenarios
│   ├── test_verifier.py   # Verifier logic + edge case tests
│   └── test_curriculum.py # Stage advancement logic tests
```

✅rl_config.yaml — Complete Parameter Reference

```yaml
horizons:
  short:  { candles: 4,   label: SHORT,  reward_weight: 1.0, punish_weight: 1.5 }
  medium: { candles: 24,  label: MEDIUM, reward_weight: 1.5, punish_weight: 2.0 }
  long:   { candles: 168, label: LONG,   reward_weight: 2.0, punish_weight: 2.5 }
  macro:  { candles: 720, label: MACRO,  reward_weight: 1.5, punish_weight: 1.0 }

decay_constants:
  SHORT:  0.035
  MEDIUM: 0.025
  LONG:   0.015
  MACRO:  0.010

decoupled_decay_multiplier: 0.5

replay_buffer:
  max_size_per_horizon:    10000
  priority_threshold:      2.0
  priority_oversample:     3
  min_per_regime:          100
  min_samples_to_train:    500

curriculum:
  stage_1: { accuracy: 0.54, reward: 0.30, min_days: 14, min_samples: 1000 }
  stage_2: { accuracy: 0.53, reward: 0.20, min_days: 21, min_samples: 500 }
  stage_3: { accuracy: 0.52, reward: 0.15, min_days: 28, min_samples: 300 }
  stage_max_wait_days: 60
```

```
self_training:
  scheduled_day:          sunday
  scheduled_hour_utc:     2
  buffer_fill_trigger_pct: 0.80
  reward_drop_trigger:    0.0
  regime_new_pred_trigger: 50
  min_cooldown_hours:     48
```

17.9 RL Monitoring Metrics

RL Monitoring Dashboard

Metric	Update Frequency	Alert Threshold	Interpretation
Rolling 24h reward score (SHORT)	Every candle	< 0.0 for 6 consecutive hours	Net punishment on short-term — investigate immediately
Rolling 7d reward score (all horizons)	Hourly aggregate	< 0.0 on any horizon	Trigger self-training if sustained; check for regime change
Directional accuracy by horizon	Daily	SHORT < 52%, MEDIUM < 51%	Deteriorating — regime change or feature drift
Mean prediction error % by horizon	Daily	SHORT > 5%, MEDIUM > 8%	Model losing magnitude precision — check feature freshness
Prediction verification rate	Every candle	< 90% of matured preds verified	Data gaps blocking verification — check OHLCV store
Curriculum stage	On stage change	Stuck > 60 days in any stage	Feature quality issue — trigger diagnostic review
Replay buffer fill %	Per verified pred	Auto-trigger at 80%	High fill = frequent self-training; monitor stability
Reward distribution histogram	Daily	Large negative tail (> 30% scores < -2.0)	Model confidently wrong frequently — urgent review
Self-training trigger count/week	Weekly	> 3 triggers per week	Too-frequent retraining = unstable — widen trigger thresholds
Flat prediction rate	Daily	> 20% of predictions FLAT	Model hedging — reduce flat calibration score

17.10 RL Error Catalogue

□ RL System Errors & Failure Modes

Error / Mistake	Where It Occurs	Consequence	How to Detect & Fix
Actual direction computed vs previous candle close (not price_at_prediction)	reward.py	Wrong direction label; model learns from corrupt signal	actual_direction = sign(actual_price - price_at_prediction). price_at_prediction is the close at T when prediction was made.
Verifier runs before candle fully closes	verifier.py	Lookahead: prediction verified against incomplete candle data	Assert: candle.close_time <= current_ts. Only verify against candles with close_time in the past.
Flat predictions dominating Replay Buffer	predictor.py	Model learns hedging is safe; stops generating directional signals	Monitor flat rate hourly. If > 20%, reduce flat calibration_score to 0.01 in rl_config.yaml.
Rare regime samples evicted from buffer	replay_buffer.py	Model forgets DECOUPLED and RANGING_LOW_VOL behaviour	Enforce min_per_regime=100. Never evict until that minimum is safe to reduce.
Self-training on interpolated candle outcomes	rl_trainer.py	Model weights updated based on fabricated price data	Verifier must check is_interpolated. Never push interpolated outcomes to Replay Buffer.
All reward scores positive (no punishment)	reward.py	Model stops learning from mistakes	Assert: < 85% of buffer scores are positive. If > 85%, reward function is misconfigured.
Curriculum advances before minimum samples met	curriculum.py	Premature promotion to harder horizon; model destabilized	Enforce min_samples per stage. Gate is: accuracy AND reward AND days AND sample_count ALL satisfied.
Model version not stored in prediction record	predictor.py	Rewards assigned to wrong model version; corrupted attribution	Write MLflow run_id into every prediction at insertion. Assert it is never NULL.
Self-training triggered more than once per 48h	rl_trainer.py	Weight oscillation; model never converges	Enforce 48h cooldown via last_trained_at timestamp. Reject trigger if within cooldown window.
Confidence score not re-calibrated post self-training	predictor.py	Confidence distribution drifts; reward function calibration breaks	After every self-training, run Platt scaling on validation set predictions. Assert mean confidence in [0.55, 0.80].

17.11 Phase 9 — RL System Build Checklist

Agent must complete all items before RL system goes live

57. Create src/rl/ directory with all 7 module files

58. Create TimescaleDB table doge_predictions with exact schema from Section 17.2
59. Create TimescaleDB table doge_replay_buffer
60. Create config/rl_config.yaml with all parameters from Section 17.8
61. Implement compute_reward() in src/rl/reward.py
62. Write unit tests: assert correct reward score for all 8 scenarios in Section 17.4
63. Implement PredictionVerifier in src/rl/verifier.py
64. Unit test verifier: assert rejection of unmatured predictions, assert rejection of interpolated candles
65. Implement Replay Buffer with regime-stratified minimum and prioritised sampling
66. Implement CurriculumManager — test that Stage 1 cannot advance unless all 4 criteria met simultaneously
67. Integrate multi-horizon predictor into inference engine: 4 predictions generated at every candle close
68. Verify Prediction Store immutability: write a test that confirms prediction fields cannot be overwritten
69. Add all RL metrics from Section 17.9 to Grafana dashboard
70. Run 7-day historical simulation: replay 7 days of DOGE candles through full RL loop
71. Assert simulation output: reward distribution contains both positive and negative values
72. Assert simulation output: verification rate > 90%
73. Assert simulation output: no FLAT prediction rate > 20%
74. Confirm self-training trigger fires correctly in simulation when buffer reaches 80%

Final Agent Directive — v3.0

DOGE is not a simple asset. It is a community-driven, sentiment-amplified, high-beta instrument that can behave rationally for weeks and then violently irrationally for days.

The self-teaching RL system transforms the model from a static predictor into a living system that improves with every verified prediction. Over time, the Prediction Store becomes a permanent record of the model's intellectual history — every prediction it has ever made, every outcome, every reward earned or punishment received. This is the model's memory.

The curriculum ensures the model earns its complexity. It must prove it can predict 4 hours ahead before it is trusted with 7 days. Competence is demonstrated before it is assumed.

The reward function is the conscience of the system. It punishes overconfidence severely and rewards calibrated certainty. A model that learns to say 'I am 80% confident' only when it is actually right 80% of the time is more valuable than any model that merely optimises a loss function on historical data it has already memorized.

The market never stops. The learning loop never stops.
The agent's mandate is to build, run, monitor, and improve this system indefinitely.

When in doubt: halt, validate, and investigate.

Silenced errors in live trading become real financial losses.